



**RAJAGIRI SCHOOL OF ENGINEERING &
TECHNOLOGY**
(AUTONOMOUS)

SECURE API COMMUNICATION SYSTEM
SIC PROJECT

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Neha Benny (U2303167)

Nikitha Praveen Basil (U2303171)

Rebecca Oommen (U2303186)

Ritu Franklin (U2303191)

Sanjana M Paul (U2303199)

Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

Contents

1	Abstract	2
1.1	Abstract	2
2	Introduction	3
2.1	Introduction	3
2.2	Objectives	3
3	System Architecture / Design	4
3.1	System Architecture / Design	4
4	Implementation	6
4.1	Implementation	6
5	Cryptographic Techniques Used	8
5.1	Cryptographic Techniques Used	8
6	OWASP Top 10 Security Analysis	10
6.1	OWASP Top 10 Security Analysis	10
7	Results / Output	12
7.1	Results / Output	12
8	Conclusion & Future Improvements	15
8.1	Conclusion	15
8.2	Future Improvements	15
9	References	16
9.1	References	16

Chapter 1

Abstract

1.1 Abstract

This project presents a Secure API Communication System designed to protect data exchange between a client and a server. The system implements Elliptic Curve Diffie-Hellman (ECDH) key exchange to securely establish a shared secret over an insecure network. To ensure data integrity and prevent tampering, HMAC-SHA256 is used to sign each request. The system also includes HTTPS-based communication using TLS to provide encryption during data transmission.

A secure authentication mechanism is implemented where user credentials are hashed using SHA-256 with a random salt. Replay attacks are mitigated by incorporating timestamps in each request and validating their freshness on the server side. Additionally, the system demonstrates resistance to Man-in-the-Middle attacks through signature verification and key exchange mechanisms. The main goal of this project is to provide a practical demonstration of modern cryptographic techniques used to secure API communications.

Chapter 2

Introduction

2.1 Introduction

In today's web-based applications, APIs are widely used to enable communication between different systems. However, most basic API implementations do not provide strong security, which makes them vulnerable to attacks such as data tampering, unauthorized access, and Man-in-the-Middle attacks. This creates a serious risk, especially when sensitive information is being transmitted.

The purpose of this project is to design and implement a Secure API Communication System that can protect data during transmission. Instead of relying only on basic security methods, this system uses a combination of cryptographic techniques to make the communication more secure and reliable.

Security plays an important role in modern web systems, where data is constantly exchanged over networks. If proper protection is not applied, attackers can intercept or modify the data without being detected. This project addresses that issue by introducing secure key exchange, authentication, and data integrity verification mechanisms.

Overall, the system aims to ensure that only authorized users can access the API, that the data remains unchanged during transmission, and that common attacks such as replay attacks are prevented.

2.2 Objectives

- To implement secure communication between the client and server using encryption techniques.
- To demonstrate a reliable authentication mechanism for verifying users.
- To ensure data integrity by detecting any unauthorized changes in transmitted data.
- To prevent common attacks such as replay attacks and Man-in-the-Middle attacks.

Chapter 3

System Architecture / Design

3.1 System Architecture / Design

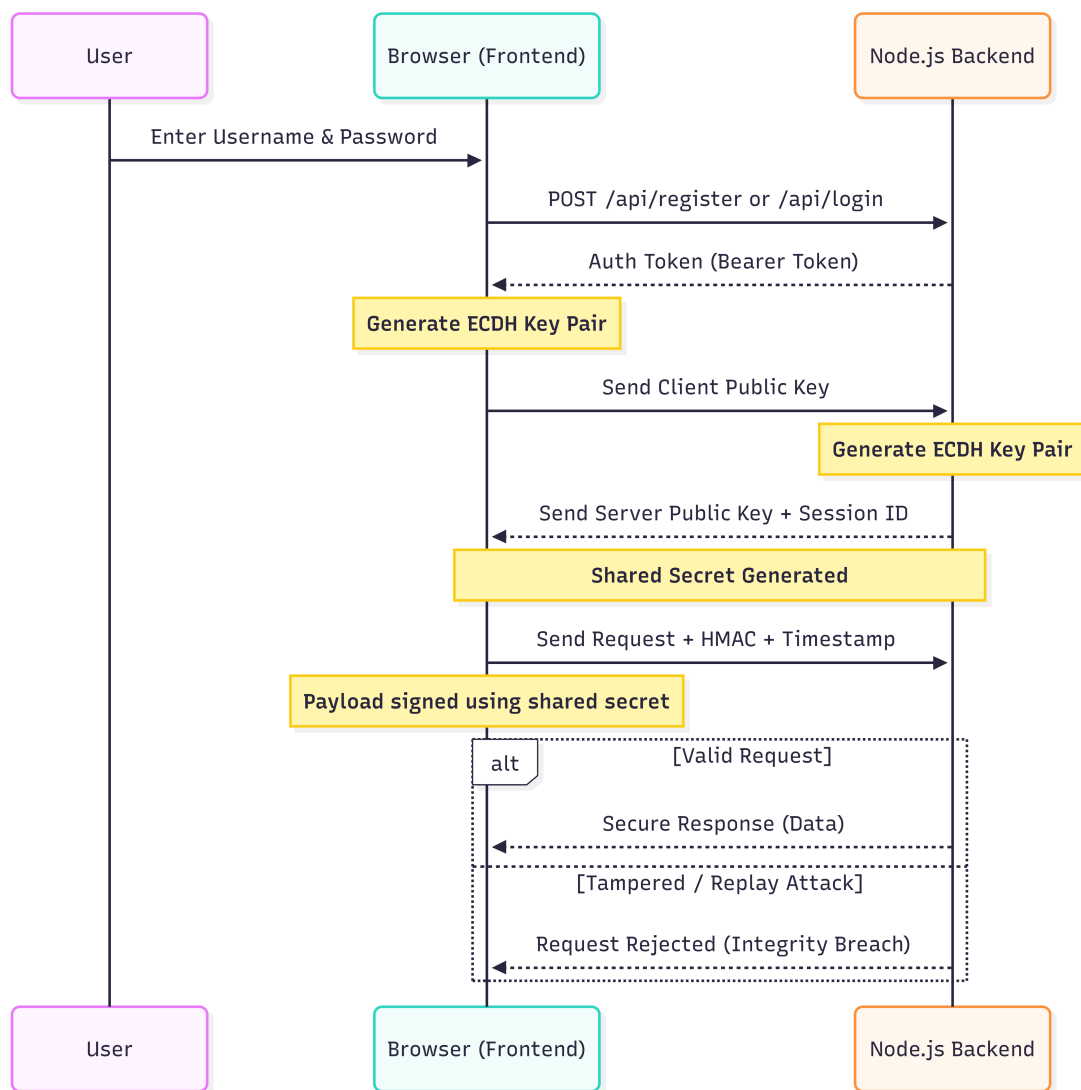


Figure 3.1: System Architecture of Secure API Communication System

The system follows a client-server architecture where a browser-based frontend communicates with a Node.js backend. The primary objective is to ensure that all data exchanged between the client and server is secure, reliable, and protected from common network-based attacks.

The process begins when the user enters their credentials through the frontend interface. These details are sent to the backend for authentication. If the credentials are valid, the server generates an authentication token, which is used to authorize all subsequent requests made by the client.

After successful authentication, a secure key exchange takes place using Elliptic Curve Diffie-Hellman (ECDH). Both the client and server generate their own key pairs and exchange only their public keys. Using this information, a shared secret is derived independently on both sides without actually transmitting it over the network.

The main components and their responsibilities in the system are as follows:

- The client (frontend) handles user interaction, generates cryptographic keys, and signs outgoing requests.
- The server (backend) performs authentication, manages sessions, and verifies all incoming requests.
- The communication channel is secured using HTTPS to ensure encrypted data transmission.

Once the shared secret is established, it is used to secure further communication between the client and server. Each request includes the actual data along with a timestamp and a signature generated using HMAC-SHA256. This ensures that the request has not been modified during transmission.

On receiving a request, the server performs multiple checks before processing it. The authentication token is validated to confirm the identity of the user. The HMAC signature is then verified to ensure data integrity, and the timestamp is checked to prevent replay attacks.

Overall, the system uses a layered approach to security, combining authentication, key exchange, encryption, and integrity verification. This design helps in protecting the system against unauthorized access, data tampering, and other common threats.

In addition to these mechanisms, the system is designed to be simple yet effective, making it suitable for understanding how modern secure communication works in real-world applications. The integration of multiple security layers ensures that even if one mechanism is compromised, others continue to protect the system.

Chapter 4

Implementation

4.1 Implementation

The Secure API Communication System is implemented as a full-stack web application consisting of a frontend and a backend. The frontend handles user interaction and performs cryptographic operations, while the backend manages authentication, key exchange, and request validation.

The system demonstrates secure communication using modern cryptographic techniques such as ECDH key exchange, HMAC-based integrity verification, and HTTPS encryption.

The implementation details are as follows:

- **Programming Language:** JavaScript is used for both frontend and backend.
- **Frontend:** Built using Vite and Vanilla JavaScript. The Web Crypto API is used for generating key pairs and deriving shared secrets.
- **Backend:** Developed using Node.js with the Express framework. The built-in `crypto` module is used for hashing, token generation, and secure comparisons.
- **Major Modules:**
 - Authentication Module – Handles user login and registration using SHA-256 with salt.
 - Key Exchange Module – Implements ECDH to establish a shared secret.
 - Request Validation Module – Verifies HMAC signatures and timestamps.
 - Secure Communication Module – Uses HTTPS for encrypted data transmission.

Sample Code Snippets:

1. Password Hashing (Backend):

```
function hashPassword(password, salt) {  
  return crypto.createHash('sha256')  
    .update(salt + password)  
    .digest('hex');  
}
```

2. HMAC Generation (Backend):

```
const hmac = crypto.createHmac('sha256', secret)
    .update(data)
    .digest('hex');
```

3. Token Verification (Security Check):

```
if (!crypto.timingSafeEqual(expectedBuf, receivedBuf)) {
    return null;
}
```

4. ECDH Key Generation (Frontend):

```
const keyPair = await crypto.subtle.generateKey(
    {
        name: "ECDH",
        namedCurve: "P-256"
    },
    true,
    ["deriveKey"]
);
```

The system also includes an attack simulation feature where request payloads can be intentionally modified. This helps demonstrate how the server detects tampering and rejects invalid requests, proving the effectiveness of the implemented security mechanisms.

Chapter 5

Cryptographic Techniques Used

5.1 Cryptographic Techniques Used

This project uses modern cryptographic techniques to secure communication between the client and the server. The system focuses on secure key exchange, authentication, and integrity verification of API requests. The main algorithms used are Elliptic Curve Diffie–Hellman (ECDH) for shared secret generation and HMAC-SHA256 for request authentication.

Encryption Algorithm: ECC (ECDH)

The project uses Elliptic Curve Cryptography (ECC) in the form of Elliptic Curve Diffie–Hellman (ECDH) with the P-256 (prime256v1 / secp256r1) curve.

During the handshake phase, the client and server generate their own key pairs. The public keys are exchanged, and both sides compute the same shared secret using their private key and the other side’s public key.

The shared secret is never transmitted directly over the network. It is stored temporarily for the session and used to sign requests.

Security provided by ECDH:

- Secure key exchange over an insecure channel
- Prevents exposure of secret keys
- Enables session-based secure communication

Hashing / Authentication: SHA-256 and HMAC

The project uses HMAC-SHA256 for authentication and integrity verification.

SHA-256 is a cryptographic hash function that produces a 256-bit digest. HMAC uses the shared secret together with SHA-256 to generate a secure signature.

Each request is signed as:

```
HMAC(secret, payload + timestamp)
```

The server recomputes the HMAC using the stored shared secret and compares it with the received signature.

Security provided:

- Integrity – detects if data was modified
- Authentication – verifies sender knows the shared secret
- Replay protection – timestamp prevents reuse of old requests

The system also uses constant-time comparison to prevent timing attacks.

Chapter 6

OWASP Top 10 Security Analysis

6.1 OWASP Top 10 Security Analysis

This project demonstrates protection against several security risks listed in the OWASP Top 10, especially those related to cryptographic failures and insecure API communication.

Cryptographic Failures / Sensitive Data Exposure

Without proper cryptographic protection, attackers may modify or forge API requests. The system prevents this using ECDH key exchange and HMAC verification.

Security improvements:

- Shared secret is never transmitted
- Every request is signed using HMAC-SHA256
- Modified data is detected and rejected

Broken Authentication

Unauthorized users should not be able to access protected resources.

Security improvements:

- User authentication required before secure operations
- Session ID used to identify valid connections
- Requests without a valid session are rejected

Replay Attacks

Attackers may capture a valid request and resend it.

Security improvements:

- Each request includes a timestamp
- Requests older than 30 seconds are rejected
- Future timestamps are also rejected

Man-in-the-Middle Attacks

Attackers may modify data during transmission.

Security improvements:

- Payload is signed using HMAC
- The server recomputes the signature
- If a mismatch occurs, the request is rejected

Timing Attacks

Timing attacks try to guess secrets by measuring response time.

Security improvements:

- `crypto.timingSafeEqual()` used for comparison
- It prevents timing-based key guessing

Chapter 7

Results / Output

7.1 Results / Output

- Screenshots of system outputs are shown below.
- Secure communication using ECDH and HMAC-SHA256 is demonstrated.
- Security tests, including attack simulations, are also shown.

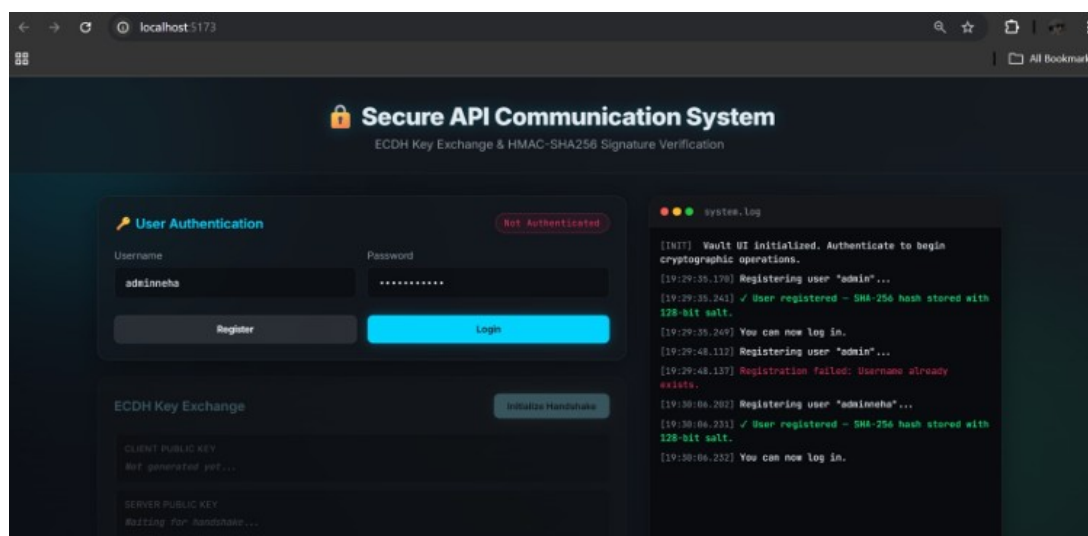


Figure 7.1: Successful user registration. The user must authenticate before accessing secure API operations.

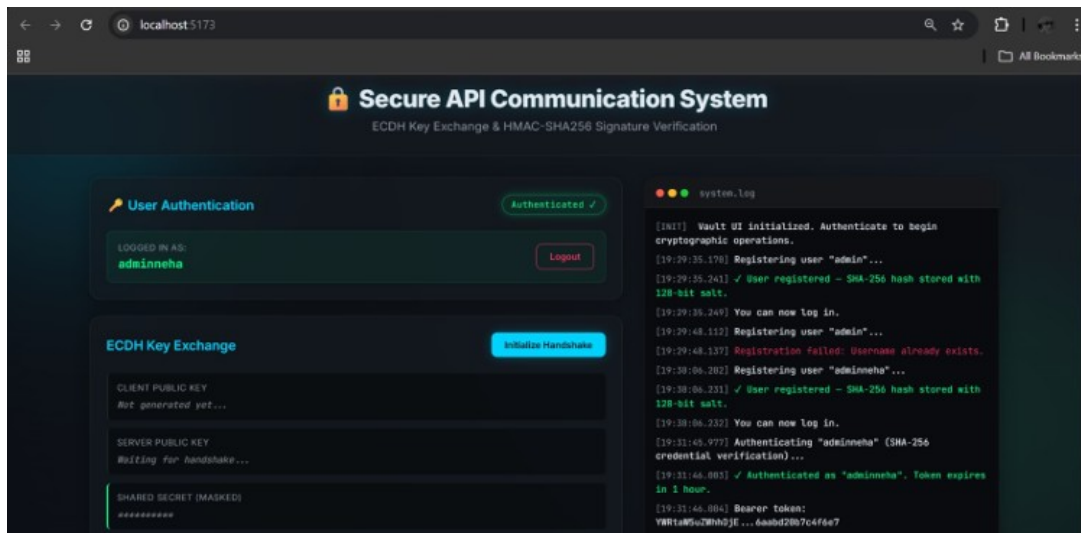


Figure 7.2: User logged in successfully. Only authenticated users can initiate secure communication.

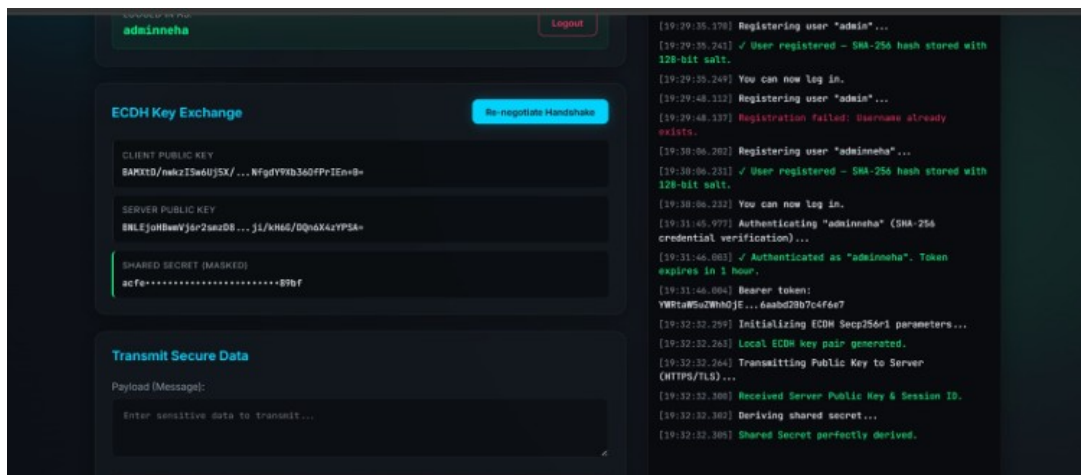


Figure 7.3: ECDH handshake initialized. Client and server exchange public keys and derive the same shared secret.

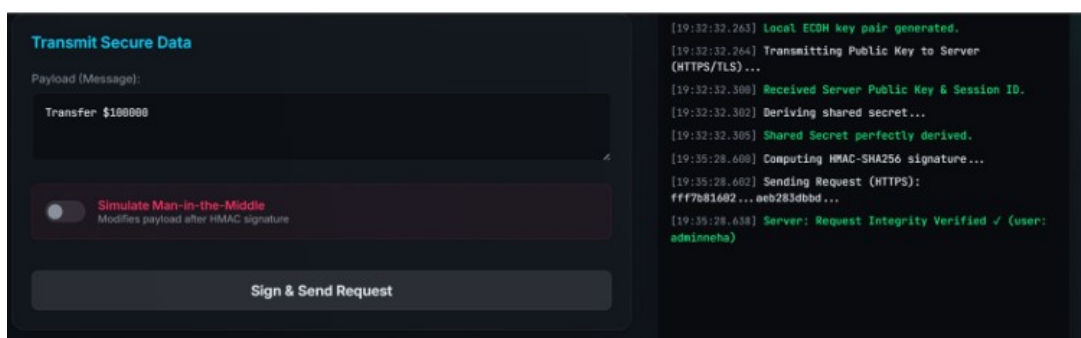


Figure 7.4: Secure request signed using HMAC-SHA256 and verified by the server using the shared secret.

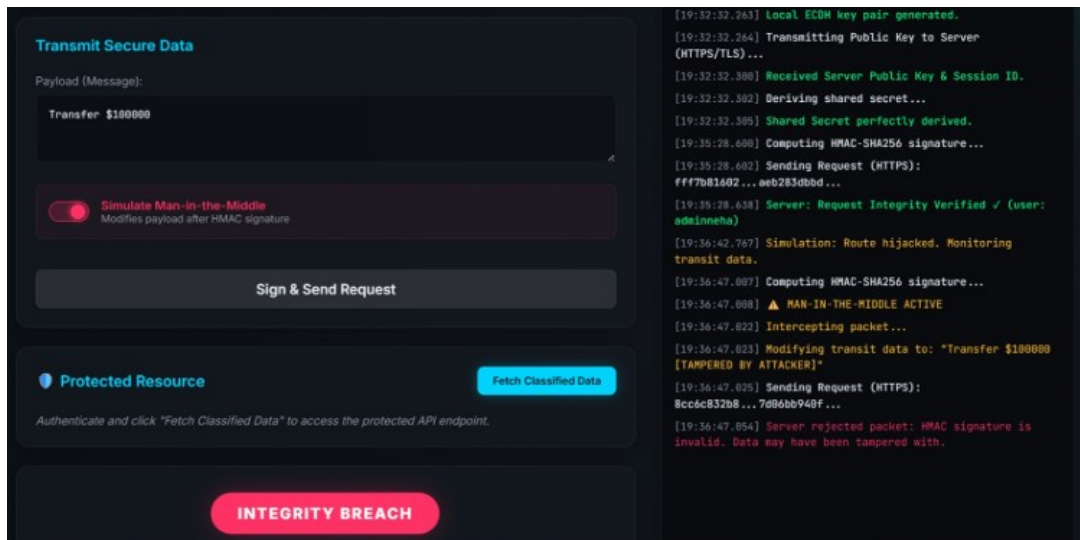


Figure 7.5: Man-in-the-Middle attack simulation. Payload modification causes an HMAC mismatch, and the server rejects the request.

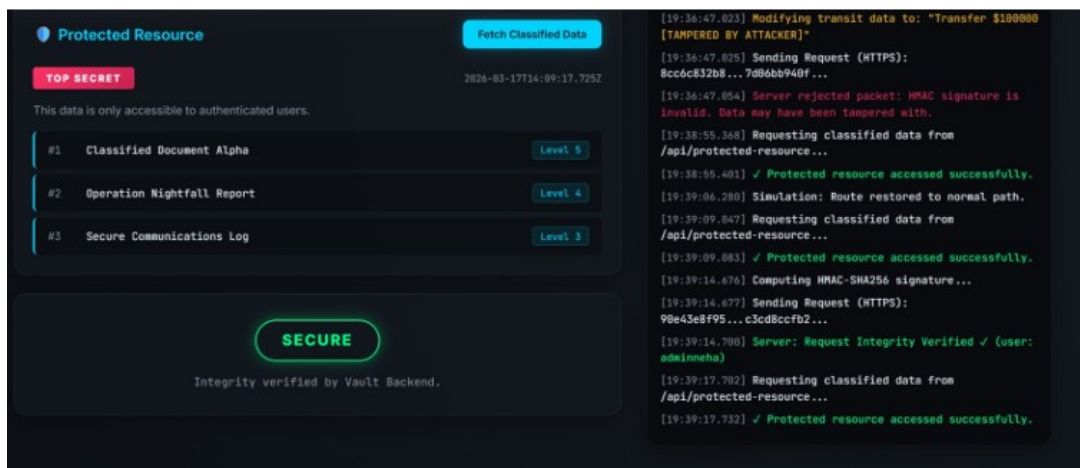


Figure 7.6: Protected resource accessed successfully after authentication and signature verification.

Chapter 8

Conclusion & Future Improvements

8.1 Conclusion

This project successfully demonstrates a secure API communication system using ECDH key exchange and HMAC-SHA256 authentication. The system establishes a shared secret between the client and server and uses it to verify the integrity and authenticity of every request.

The project shows how cryptographic techniques can protect API communication from tampering, replay attacks, and unauthorized access. The implementation includes handshake validation, request signing, timestamp checking, and constant-time comparison for secure verification.

This prototype demonstrates the importance of combining key exchange, hashing, and authentication to build secure web systems.

8.2 Future Improvements

- Add AES encryption to provide full confidentiality of payload data
- Deploy the system on a real cloud server using HTTPS
- Add multi-factor authentication for stronger security
- Store sessions in a database instead of memory
- Implement automatic session expiration

Chapter 9

References

9.1 References

- OWASP Foundation – OWASP Top 10 Web Security Risks
- OWASP API Security Top 10
- NIST Secure Hash Standard (SHA-256)
- RFC 2104 – HMAC: Keyed-Hashing for Message Authentication
- Node.js Crypto Module Documentation
- MDN Web Crypto API Documentation
- Project Presentation – Secure API Communication System